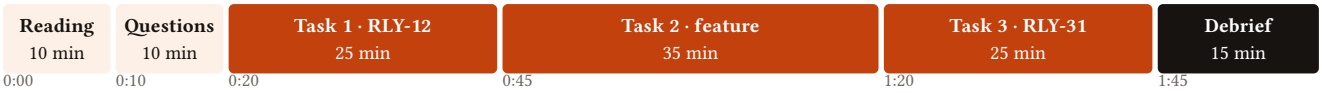
INTERVIEWER GUIDE

AI Session: Relay

The session at a glance

Two hours, five blocks. Keep this page next to you; everything else in this guide is detail for one of these rows.



Clock	Block	You do	Free periods
−0:15	Before	Start examples/cluster.sh on the candidate's machine and check one curl. Note their AI tool, skills, MCP servers and rules file. Ask them to turn web and GitHub access off. Cards in order: reading, RLY-12, the feature(s), RLY-31.	—
0:00	Welcome	Read "What you hand to the candidate" aloud. Hand the reading card. Start the timer.	—
0:00–0:10	Reading	Watch. Note what they open themselves and what they ask the AI. No hints.	**History**
0:10–0:20	Questions	AI closed, editor open. Ask 1, 2, one flow question (11 or 12) and three or four others. Note answers; correct nothing.	none: you are asking
0:20–0:45	Task 1	Hand the RLY-12 card. Note whether they reproduce before they fix. At 0:40: "five minutes, then tag".	**Projects**
0:45–1:20	Task 2	Hand the feature card(s) you chose. At 1:15: "five minutes, then tag".	**Way of working**
1:20–1:45	Task 3	Hand the RLY-31 card. At 1:40: "five minutes; write the incident note now".	**AI setup**
1:45–2:00	Debrief	AI closed. Always question 2; then as many others as fit.	none
after	Wrap up	Collect the repository with tags, the AI transcript and the recording. Fill in the ratings the same day, from the bug sheet and your notes.	—

Free periods are the minutes when the candidate is waiting for the AI to finish. Use them for the conversation topic of that block (see "While the AI works"), and stop as soon as the output arrives. A block with no waits gets no conversation; that is fine.

If a task ends early, ask its one follow-up probe (in "While the AI works"), then hand the next card. Spare minutes go to the next task, never to the debrief. **If you run late,** stop the task at its time box anyway and have them tag; if Task 1 overran by more than five minutes, give a Level 1 feature in Task 2; never give Task 3 less than 20 minutes or the debrief less than 10.

What this session measures

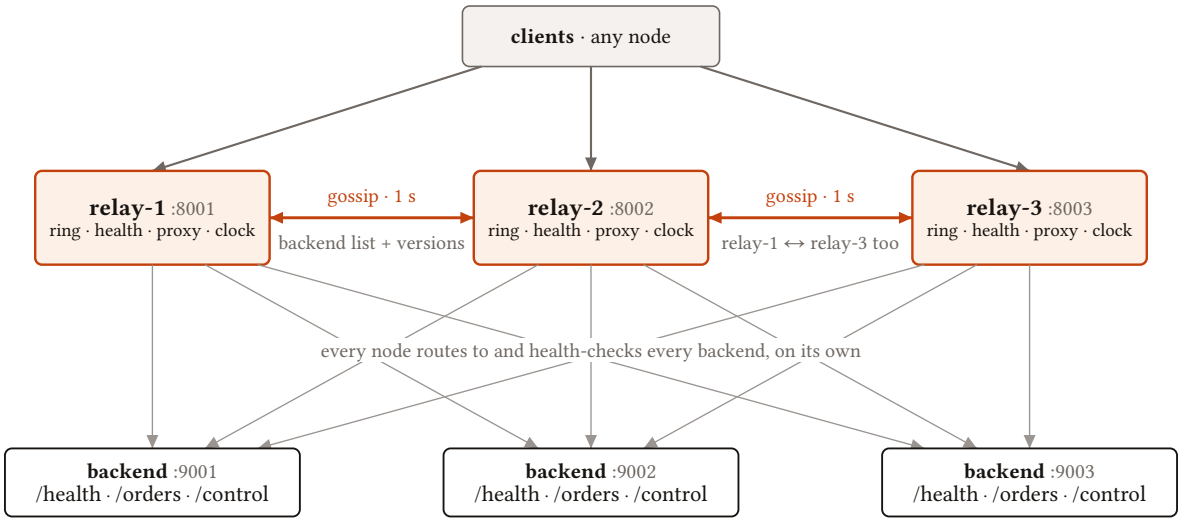
A two-hour session on a small codebase the candidate has never seen, with an AI assistant of their choice. They first read the code with the assistant and answer our questions about its packages, its functions and its flow. Then they fix one bug we tell them about, build one or two features you pick from a menu, and work through an incident report whose causes we do not tell them. Whether they finish does not matter much. What matters is how they work with the assistant, and whether they notice when the assistant is wrong.

Why this codebase

Relay is a small distributed HTTP load balancer, written in Go with nothing but the standard library. Several Relay nodes run side by side; each can take any request. They agree on the list of backends by gossip, every change carrying a version from a logical clock, and they route with a consistent-hash ring, so every node sends the same client to the same backend.

Each node health-checks every backend on its own and retries a failed request on the next backend of the ring. About 1,000 lines of code including a demo backend that can be told to fail, and 500 lines of tests.

The ideas are ones every backend engineer has heard of: consistent hashing, health checks, retries, gossip, version vectors. That is exactly what makes it a good test. An AI assistant recognises each pattern at a glance and describes the textbook version of it, and the code in front of the candidate is not quite the textbook version. The distance between "I know what a Lamport clock is" and "this code does not advance it on receive" is the distance this session measures.



Each node is the same program. The only thing they exchange is the backend list, by gossip; health, the ring, the clock and every request's retry state stay inside one node.

Everything runs on one laptop: examples/cluster.sh starts three backends and three nodes, and every claim can be checked with curl in seconds. Whether the candidate does is the main thing we watch.

The code carries six bugs that the tests do not catch. All six were reproduced on a running cluster and with failing tests before this guide was written; the numbers below come from those runs.

	Bug	Where	Effect	Surfaces in
B1	The ring walk stops after as many points as there are backends	ring · Lookup	With one backend of three down, 1–6% of keys get 503 "no backend available" although two backends are healthy	Task 3, F5
B2	A success never resets the failure count	health · Record	Failures add up across weeks; a backend that once had trouble is ejected by a single failed check	Task 3, F4
B3	The logical clock is not advanced on receive	cluster · Merge	A change made through a node that only received versions gets a lower version and is reverted by gossip	Task 1, F3
B4	Every method is retried after a 5xx	proxy · ServeHTTP	A POST the backend already handled is sent to a second backend: duplicate orders	Task 3
B5	The client's address is taken with its port	proxy · ServeHTTP, forward	Clients without X-Relay-Key are not sticky across connections; X-Forwarded-For carries the port and overwrites the incoming chain	Reading, F2
B6	Removing a backend deletes its entry	cluster · Remove	The next gossip round brings it back from a peer, on every node	Task 1 (reported)

The README and the package comments describe the intended behaviour in plain words: "marked down after 3 failed checks **in a row**", "each backend is tried at most once", "the client's IP address". Each of those sentences is true of the design and false of the code. A candidate who turns a sentence of the README into a test finds a bug.

Format

Individual, about 120 minutes a reading phase, three tasks, then a 15 minute debrief. Book a two-hour slot.

Read first, then questions the session opens with 10 minutes of reading the code with the AI, then 10 minutes of questions about its packages, its functions and the flow of a request through them, with the AI closed. Nobody changes code before the questions are done.

Talk while the AI works when the candidate is waiting for the AI to generate code, use the time to talk about their history, their projects and their way of working. Stop as soon as the output arrives. See "While the AI works" below.

Their own AI any tool is allowed; Claude Code, Cursor, Copilot, or a plain chat window. Note which one it was and whether it could run commands by itself. Note before the session which skills, MCP servers and rules files (CLAUDE.md, AGENTS.md, Cursor rules) are configured. A web search or GitHub MCP server can find public code, and this repository is public and contains this guide: ask the candidate to turn web and GitHub access off for the session, or note that it stayed on.

Go only there is one implementation. A candidate who does not write Go can still take the session, since the code is small and plain, but say so on the sheet; the Coding rating then weighs judgment over idiom.

Observed one interviewer with a notes sheet, plus screen recording. Without an observer, the AI transcript and the git history keep most of what you need.

Commit after every task the candidate tags the end of each task (task-1, task-2, …). Unfinished work is fine and is evidence in itself.

Tasks are handed out one at a time Task 1 is the way into B3, and the feature you choose may surface B1, B2 or B5 before Task 3 does. Handing tasks out early spoils that.

Preparation

- [ ] Copy ai-session/relay/ from this repository into a **new, private** repository with a neutral name (for example relay) and a single initial commit. Do not link to this repository: it is public and contains this guide.
- [ ] Do **not** fix anything. Every task depends on the bugs being there.
- [ ] Go 1.27 or later, curl, and python3 or jq for reading JSON. Ports 8001 to 8003 and 9001 to 9003 must be free.
- [ ] Confirm the setup: go test -race ./... passes; examples/cluster.sh starts; curl -H 'X-Relay-Key: alice' localhost:8001/hello and the same on :8002 name the same backend; the RLY-12 steps below bring the deleted backend back.
- [ ] Choose the feature(s) for Task 2 from the menu, according to the level you are hiring for and the time you expect to have.
- [ ] Print this guide and write each brief on a separate card.

What you hand to the candidate

> **Read verbatim:** You are joining the team that maintains Relay, a small distributed load balancer: several nodes that share a list of backends and route requests to them. You have never seen this code. You have about two hours and an AI assistant of your choice.
>
> 1. Use the AI as much or as little as you like. We care about how you work, not about whether you use it.
> 2. Think aloud. Tell us what you ask the AI and why, and what you conclude after it answers.
> 3. After each task, commit and tag (git tag task-N). Unfinished is fine.
> 4. If something in a task is unclear, you may ask us. We may or may not answer.
> 5. Keep your AI transcript. We collect it at the end.
> 6. While your AI is generating code, we may talk about your background and how you work. Go back to the output whenever you want; you do not need to finish the sentence.

While the AI works

An agentic tool often works for a minute or more before it shows anything. Do not spend that time in silence: talk with the candidate about their history, their projects and their way of working. It fills dead time, and it gives us a second view of the person next to the one the tasks give.

Rules:

- Talk only while the candidate is **waiting**. Never interrupt them while they are reading output, writing a prompt or thinking aloud about the code.
- Stop as soon as the output arrives, even mid-answer. What the candidate does next is evidence: do they turn to the output and read it, or keep talking and accept it unread? Note which.
- Do not talk about the task, the bugs or the code. That is what the debrief is for, and a hint here spoils the tasks.
- The time boxes do not stop for the conversation. Keep the questions short, and pick up a topic again at the next wait instead of finishing it.
- Ask only about work. No questions about age, family, health, religion or anything else that has no place in an interview.

Each block has its own topic, so the conversation moves through the candidate's story in order and you never have to think of a question on the spot. Go down the list; tick each question you asked; when a topic runs out before the block does, borrow from the next one. You will not get through all of them, and that is fine.

Block	Topic	Questions, in order	Write down
Reading 0:00–0:10	**History**	1. How did you get into software? 2. Which job or project taught you the most, and what? 3. What bug did you chase the longest, and how did you find it in the end?	How they explain a past problem: cause, or story?
Task 1 0:20–0:45	**Projects**	1. Tell us about a system you work on now. What is your part of it? 2. Has it run behind a load balancer or with service discovery? What went wrong with it? 3. Which decision in it would you make differently today? 4. When were you last on call, and what was the incident?	Can they draw their own system in words? Do they know what is distributed in it?
Task 2 0:45–1:20	**Way of working**	1. How do you use an AI assistant day to day, and what do you not use it for? 2. When did an AI last give you something wrong, and how did you notice? 3. How do you review a teammate's code? What do you look at first? 4. When do you write tests: before, during or after? 5. How do you pick up a codebase you have never seen?	Anything the session contradicts, such as "I always read the diff" followed by an unread accept.
Task 3 1:20–1:45	**AI setup**	1. Which skills and MCP servers have you configured, and why those? 2. Is there one you wrote yourself? Show it. 3. Which did you try and remove, and why? 4. What is in your rules file? 5. Which of these did your AI use today, and did it help?	Whether they know what their setup does, not how big it is.

Early finish, one probe per task, then hand the next card:

Task 1 "relay-3 was switched off during the delete. Show me that it ends up right when it comes back." (If they have not yet re-added through another node, ask for that instead.)

Task 2 "Show me the test that would fail if two requests updated your state at the same time." Or the second feature of the same level.

Task 3 "Write the one test that would have caught the 503s." (With any one backend down, every key is served.)

A setup is not scored by its size. A candidate with a plain tool and no configuration loses nothing. What counts is whether they know what their setup does and why it is there: a skill they cannot explain, or an MCP server they did not know was on, is worth a note.

Score this conversation in the same four competencies as everything else; do not give it a score of its own. Most of it is evidence for **effective communication**: can they explain a system they know to someone who does not? An answer about their way of working that the session itself contradicts (says they always read the diff, then accepts output unread) is worth a note too.

READING

Read the code

20 minutes

BRIEF FOR THE CANDIDATE

Read the code with your AI assistant, in any way you like: ask it for an overview, go through the files together, run the tests, start the cluster with examples/cluster.sh and send it requests. Do not change any code. You have 10 minutes.

Then we close the AI and ask you about the packages, the functions, and how a request flows through them. The code stays open, and you may look at it while you answer.

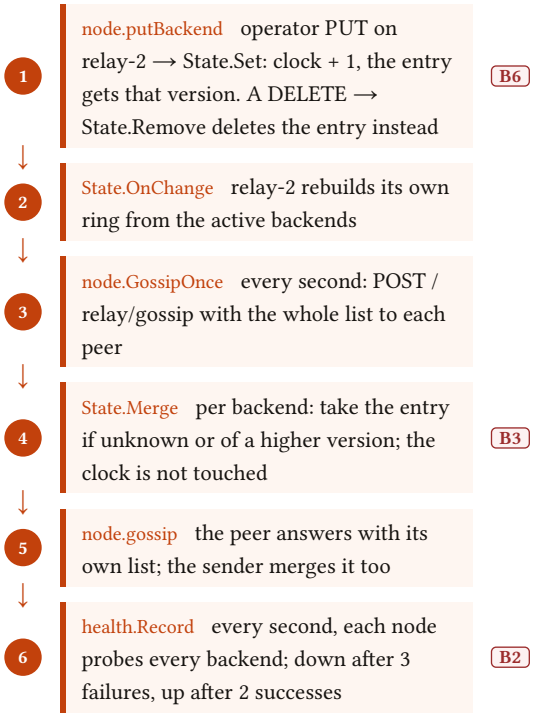
No trap. This is the baseline for how the candidate reads with the AI, and the questions show what they took away from it: their own picture of the code, or the assistant's summary. For the questions, the AI window is closed or minimised; the editor stays open. Finding the answer in the code is a good answer. Guessing from what the AI said, without looking, is the thing we want to see.

Keep the two pictures below in front of you while you ask. Each step names where it happens in the code; the tags show where a bug lives, so you can tell which answers "touch" one.

A request (question 11)



A change, and health (questions 5, 6, 12)



Ask six or seven of the questions below: always 1 and 2, and at least one flow question (11 or 12). After each answer ask "where is that?" and let them point at the code. Do **not** say whether an answer is right, and do not correct it: Tasks 1 to 3 depend on the candidate finding the bugs themselves. If they name a bug now, write it down and move on.

	Question	What a good answer says
1	What are the packages, and what does each one do?	ring places backends on a consistent-hash ring and finds a key's backend; health checks backends with GET /health; cluster keeps the versioned backend list; proxy forwards a request, picks the affinity key and retries; node wires them together and serves the admin API and gossip; cmd/relay starts a node, cmd/backend is the demo service.
2	What do the nodes share, and what does each node keep to itself?	Shared by gossip: the backend list, each entry with a status and a version. Not shared: health (each node checks on its own), the logical clock, the ring (built locally from the active backends, so it is the same only when the lists agree), and the retry state of a request.
3	How does a request choose its backend?	The key is X-Relay-Key, or else r.RemoteAddr; node.pick hands it to ring.Lookup, which hashes it, binary-searches the first point at or after the hash, and walks clockwise to the first point whose backend is healthy and not tried yet. A candidate who looked at RemoteAddr says it includes the port. **Touches B5.**
4	What does Lookup do when the key's backend is down?	It moves on to the next point. The loop runs len(r.backends) times, so it looks at as many **points** as there are backends; a down backend often owns several points in a row. Only a candidate who read the loop, rather than its comment, says this. **Touches B1.**
5	When is a backend marked down, and when up again?	Down after Fall = 3 failures, up after Rise = 2 successes, by Record. A success resets nothing but successes; a failure resets successes. Only a candidate who read Record notices that failures is never reset by a success. **Touches B2.**
6	An operator drains 9002 through relay-2. What happens on relay-2, and how does relay-1 learn about it?	putBackend calls State.Set, which increments the node's clock and stores the entry with that version; OnChange rebuilds the ring without 9002. On the next gossip round relay-2 posts its list to each peer; the peer's Merge keeps the higher version, and its own OnChange rebuilds its ring. The answer also comes back the other way (push and pull). **Touches B3:** the clock moves only in Set.

	Question	What a good answer says
7	What does DELETE /relay/backends/... do, on this node and on the others?	It deletes the entry from the local map and rebuilds the ring. Nothing is sent to the other nodes. **Touches B6;** Task 1 is about this, so do not push further.
8	When does the proxy retry, and where does the retry go?	After a transport error or a 5xx, when it was not the last attempt; there is one retry by default. The retry goes to the next usable backend on the ring, the first one being marked as tried. The body was read into memory first so it can be sent again. Any method is retried. **Touches B4.**
9	What do backends see in X-Forwarded-For?	r.RemoteAddr, set, not appended: an incoming chain is lost, and the value carries the port. **Touches B5.**
10	A new backend is added through relay-1. When does relay-3 start sending it traffic, and is it health-checked first?	After a gossip round reaches relay-3 (relay-1 pushes, or relay-3 pulls), within about a second. It is not checked first: Healthy treats a backend it has never checked as healthy.
11	Flow: follow curl -H 'X-Relay-Key: alice' localhost:8001/hello to the backend and back.	The mux sends /hello to the proxy; ServeHTTP reads the body through MaxBytesReader, takes the key, calls Pick (the node's pick, which reads the current ring under the read lock) with a usable that checks tried and Healthy; forward builds a new request to http://backend/hello, clones the headers without hop-by-hop ones, sets X-Forwarded-For, X-Forwarded-Host and X-Relay-Node; reply copies status, headers and body back and adds X-Relay-Backend.
12	Flow: what happens on relay-2 when relay-1's gossip arrives?	POST /relay/gossip hits node.gossip, which decodes the list, calls Merge (per backend: take the remote entry if unknown or of a higher version), triggers rebuild through OnChange if anything changed, and answers with relay-2's full list, which relay-1 merges in exchange.

What to watch for:

- While reading: do they only ask the AI for a summary, or do they open files, ask for line references, run the tests, and start the cluster?
- Are the answers specific (a file, a function, a field) or general ("it uses consistent hashing")? Does the candidate look at the code before answering, or answer from the AI's summary? The summary will say "Lamport clock", "sticky by client IP" and "N consecutive failures". All three are what the comments say, not what the code does.
- Wrong answers that repeat a claim the AI made are the main evidence here. Note each one; they count as AI claims accepted without checking.
- "I don't know, let me look" is a good answer. A confident guess is not.

TASK 1 The backend that would not leave (known bug)

25 minutes

BRIEF FOR THE CANDIDATE

Ticket **RLY-12**, from operations:

"We decommissioned 127.0.0.1:9003 with DELETE /relay/backends/127.0.0.1:9003 on relay-1. The call returned 204. A second later the backend was back in /relay/status on every node, relay-1 included, and still taking traffic. In the end we had to stop the machine."

Fix it, with a regression test that fails before the fix and passes after it.

A removed backend must stay removed on every node, including a node that could not be reached when the delete happened. The same address must be addable again later, through any node.

```
examples/cluster.sh
curl -X DELETE localhost:8001/relay/backends/127.0.0.1:9003      # 204
sleep 2
curl -s localhost:8001/relay/status                              # 9003 is back, active, version 0
```

Trap · a delete is not a fact that gossip can carry. Remove deletes the entry. On the next exchange relay-1 receives the peers' lists, which still hold 9003 at version 0; Merge sees an unknown backend and takes it. The fix is a **tombstone**:

an entry with status removed and a new version, gossiped like any other entry, left off the ring and out of the health checks. It is the only fix that meets the second sentence of the brief.

Fixes the AI proposes that do not meet it: sending the delete to every peer (a peer that is down misses it and brings the backend back when it returns); ignoring merged entries that are not in the seed list (no backend could ever be added again); a local "removed" set without a version (it never spreads, and it blocks the re-add for ever). Each of these passes a two-node test in which every node is reachable.

Trap · the re-add exposes B3. With a tombstone in place, removing through relay-1 and adding again through relay-2 still fails. relay-1's tombstone has version 1. relay-2 merged it but did not advance its own clock, which is still 0, so its re-add also gets version 1. Equal versions do not replace each other, so relay-2 sends traffic to the backend and relay-1 and relay-3 do not, for ever. We reproduced it in a test with the tombstone fix and without the clock fix. The fix is one line in Merge: s.clock = max(s.clock, e.Version) for every received entry.

Without the tombstone, B3 shows on its own: drain 9002 and activate it again through relay-1 (versions 1 and 2), then drain it through relay-2. relay-2 records version 1, and the next gossip round sets it back to active at version 2 on all three nodes. Reproduced on the cluster.

What to watch for:

- Does the candidate reproduce RLY-12 on the cluster before asking the AI to fix it? The steps are in the brief.
- Does the regression test involve two states that gossip with each other? A test on one State ("Remove, then the backend is gone") passes on the buggy code.
- Do they test the re-add **through the other node**? That is the only test that finds B3. If the AI writes the re-add test on the same node, it passes.
- The brief's middle sentence, "a node that could not be reached", is the requirement that rules out broadcasting the delete. Does the candidate read it as a requirement, or does the AI's broadcast pass unexamined?
- Do removed backends stop being health-checked? CheckAll takes State.Backends; if tombstones are in that list, every node keeps probing a machine that is gone. Small, and a sign of having followed the data.
- Strong candidates raise two questions by themselves: when may a tombstone be dropped (keep both for the debrief), and what happens when two nodes change the same backend at the same version.

TASK 2 Features (interviewer's choice)

35 minutes

Pick one feature, or two short ones, from the menu. The level describes the nature of the work, not the job title. Each feature is designed to bring a different part of the code into play; the last column says which bug it is likely to surface. Tell the candidate only the brief.

	Feature	Level	Time	Surfaces
F1	Request counters in /relay/status	1	15 min	data races, B4
F2	Cookie affinity	1	15 min	B5
F3	Backend weights	2	20 min	B3, B1
F4	Passive ejection	2	20 min	B2
F5	Bounded loads	3	35 min	B1

F1 Request counters

Level 1 · 15 minutes

BRIEF FOR THE CANDIDATE

GET /relay/status should show, for each backend, how many requests this node has sent to it and how many of them failed (a connection error or a 5xx) since the node started. Tests included.

What to watch for:

- The counters are written from every request's goroutine and read by the status handler. Does the candidate use atomics or a lock, and do they run go test -race with concurrent requests to show it? The README says to run the tests with -race; does the AI?

- "Requests sent to it" is per attempt, not per client request: a retried request counts once on each backend. Does the candidate notice that, and say which they chose? This is the closest F1 comes to B4.
- Where do the counters live? On the proxy, keyed by backend, is natural. In `health.Checker` mixes two things. A new map in `node` that the proxy calls into is fine.
- What happens to the counters of a removed backend?

F2 Cookie affinity

Level 1 · 15 minutes

BRIEF FOR THE CANDIDATE

Browsers cannot send `X-Relay-Key`. When a request has neither `X-Relay-Key` nor a `relay_key` cookie, Relay should set a `relay_key` cookie with a new random value on the response, and use the cookie as the key from then on, so that a browser stays on one backend. Tests included.

Trap · the first request. If the first request is routed by the client's address and the cookie is generated afterwards, the second request (routed by the cookie) usually lands on a different backend. The new value must be generated **before** picking, and used as the key of the first request too.

The candidate also has to read the fallback that is there now, `key = r.RemoteAddr`, and that is where B5 is. A candidate who tries it with `curl` and no header sees requests spread over all three backends, because every `curl` opens a new connection from a new port.

What to watch for:

- `crypto/rand`, not `math/rand`, for a value the client will keep. A cookie that is not `HttpOnly` or has no `Path` is worth a note, not a failure.
- The backend may set cookies of its own. `w.Header().Set("Set-Cookie", ...)` in `reply` would overwrite them; `Add` (or `http.SetCookie`) keeps both.
- Does the cookie travel on to the backend? Either is defensible if stated.

F3 Backend weights

Level 2 · 20 minutes

BRIEF FOR THE CANDIDATE

An operator can give a backend a weight from 1 to 10: `PUT /relay/backends/{backend}` with `{"status": "active", "weight": 3}`. A backend with weight 3 should get about three times the keys of a backend with weight 1. Weights spread to all nodes, like the status does. Tests included.

Trap · the upgrade, and the clock.

- During a rolling upgrade, old nodes gossip entries without `weight`. JSON decodes that as 0, and a backend with zero points disappears from the new nodes' rings. Missing or 0 must mean 1. Ask the candidate "what happens while half the nodes run the old version?" if they do not ask it themselves.
- Weight changes are versioned changes, so they meet B3 if Task 1 did not fix it: a weight set through a node that has only received versions is reverted by the next gossip round.
- The point names. If the replicas of a backend are named `b#0` to `b#(64w−1)`, raising a weight only adds points, and only keys near the new points move. Hashing the weight into every name (`b#w#i`) moves every key of that backend. Does the candidate test how many keys move?
- A heavy backend owns long runs of neighbouring points, so B1 hits more keys when a heavy backend is down.

F4 Passive ejection

Level 2 · 20 minutes

BRIEF FOR THE CANDIDATE

If five requests in a row that a node sends to a backend fail (a connection error or a 5xx), that node stops sending it traffic for 10 seconds without waiting for the health check. Then the backend takes traffic again. Tests included, without sleeping for 10 seconds.

What to watch for:

- The obvious move is to copy the counting in health.Record, and with it B2: a success does not reset the failures, so "five in a row" becomes "five ever". A candidate who writes the test "fail, succeed, fail, succeed, … never ejected" finds the bug in their own code and, if they look, in Record too.
- Counters written from many goroutines; -race again.
- "Without sleeping" means the time has to come from somewhere the test controls: a now func() time.Time field, or a clock interface. Does the candidate introduce one, or does the AI write time.Sleep(10 * time.Second) anyway?
- Ejection and health are separate verdicts that both have to say yes. Where is that combined: in node, in the proxy's Healthy, or by making the checker know about traffic?

F5 Bounded loads

Level 3 · 35 minutes

BRIEF FOR THE CANDIDATE

A popular key can overload its backend. On each node, no backend may have more than ⌈1.25 × average⌉ requests in flight, where the average is taken over the active, healthy backends and counts the new request. A request whose backend is full goes to the next backend on the ring that is not full. Keys must still go to their own backend whenever it is not full. Tests included.

What to watch for:

- This is "consistent hashing with bounded loads" (Mirrokni, Thorup and Zadimoghaddam, used in HAProxy and Envoy). A candidate who knows it says so; one who does not should still reason out why the walk always ends: the bound is above the average, so some backend is always below it.
- The walk past full backends is Lookup's walk, and B1 caps it at three points. With bounds in place and the cluster under load, requests start failing with 503 that the bound says must succeed. A candidate who writes the property test ("under any load, every request is served") finds B1; one who tests a single overloaded key does not.
- In-flight counts must go down on every path: success, error, retry, client gone. A defer right after the increment does it. Does the retry release the first backend before taking the second?
- Deterministic tests need requests that stay in flight: handlers that block on a channel. A test that relies on time.Sleep to "probably" overlap is flaky.

TASK 3 Tuesday's incident (unknown causes)

25 minutes

BRIEF FOR THE CANDIDATE

Ticket **RLY-31**, an incident report:

- At 14:02 a bad build of the order service went out on backend-2 (127.0.0.1:9002). Until the rollback at 14:08 it answered about half of all requests with 500, and its /health failed now and then.
- The payments team found 41 orders recorded twice in that window. In every pair, one order is on backend-2 and one on another backend, same customer, same cart, same second. Each of those customers saw one successful response.
- Support has nine tickets from customers who got "no backend available" during the window. backend-1 and backend-3 were healthy throughout.
- Since the rollback, backend-2 drops out of rotation for a couple of seconds a few times a day. The health logs show backend-1 and backend-3 failing a single check now and then too, and they never drop out.

Find out what went wrong, fix what is broken in Relay with regression tests, and write two or three sentences for the incident report on what the load balancer did wrong.

To replay it on the cluster (on our runs, 30 POSTs gave 5 duplicate orders, every client saw 200):

```
curl -X PUT localhost:9002/control -d '{"fail_rate": 0.5}'
for i in $(seq 1 30); do
  curl -s -o /dev/null -w '%{http_code}\n' -X POST -H "X-Relay-Key: user-$i" localhost:8001/orders -d "{\"cart\":$i}"
done
curl -s localhost:9001/orders; curl -s localhost:9002/orders; curl -s localhost:9003/orders
```

Trap · three symptoms, three bugs.

- **Duplicates, B4.** The demo backend does the work and then fails, as a real service does when the write succeeded and the response did not. The proxy retries any request after a 5xx, so the POST runs on a second backend too. The fix is a retry policy: retry a non-idempotent request only if it was never sent (a refused connection), and retry GET, HEAD and OPTIONS after a 5xx as before. Turning retries off entirely makes `TestRetriesOnAnotherBackend` fail, which is the right outcome: that retry is safe.
- **"No backend available", B1.** While backend-2 was out of rotation, `Lookup` gave up after three points. With 9002 failing its health check, 16 of the keys `user-0` to `user-999` got 503 on every try; `user-157` is one. The same cap hits retries harder: the retry excludes the backend just tried as well.
- **Flapping, B2.** During the incident backend-2 failed some checks without being marked down; those failures were never reset. After the rollback, the count sat at 1 or 2, so one failed check ejects it. On our cluster, after six seconds of failing health on 9002, one blip on 9002 and 9003 together ejected 9002 and not 9003.

What to watch for:

- Does the candidate reproduce each symptom before fixing it? All three can be reproduced with the demo backend's `/control` in a minute. The ticket is the only description they have; the AI has none at all.
- Expect the AI to blame the backend for the duplicates ("the order service should be idempotent; add an idempotency key"). That is good advice and not the answer: the load balancer sent the same POST twice. Does the candidate separate what Relay did wrong from what the service could do better?
- Expect the AI to explain the flapping with timeouts or with `Fall` being too low, and to propose raising `Fall`. That hides B2 for a while. Does the candidate ask why backend-1 and backend-3 behave differently under the same blips?
- For the 503s the AI often says "all backends were marked down at some point". The ticket says otherwise. Does the candidate find a key that fails and keep it as a test?
- Is the incident note honest and specific (retries on POST, the ring walk, the health counter), or does it blame the bad build alone?

Debrief (15 minutes)

1. Which state do the nodes share, and which does each keep to itself? Two nodes disagree about whether 9002 is healthy: what does a client see?
2. Which of the AI's claims turned out to be wrong today? How did you find out?
3. When may a tombstone be dropped? relay-3 was switched off for a month and comes back with the seed list: what happens? (Its seed entries are at version 0, and the peers' tombstones win, as long as the tombstones are still there. Dropped too early, the removed backend comes back from relay-3.)
4. Two operators change the same backend at the same moment through two nodes, and both changes get the same version. What does the cluster end up with? (Two different entries with the same version never replace each other: the nodes disagree for ever. A tie-break, such as the node id or an order on the statuses, settles it.)
5. A new backend counts as healthy before its first check. Is that the right default?
6. Which change would you merge with the least confidence, and why?
7. What one test would have caught B1? (With any one backend down, every key is served.)

Scoring

Score the session with the **same four competencies** and the **same scale** as the evaluation document: **Weak · Borderline · Strong · Not assessed**. Do not invent a per-task score. Each competency gets one rating for the whole session, backed by the evidence you wrote down; a competency you saw no evidence for is **Not assessed**, not a guess. Write next to the ratings which features you chose and their level, because Strong on a Level 1 feature and Strong on F5 are different evidence.

Competency	Weak	Borderline	Strong
Data structures, algorithms and problem solving	Patches each symptom where it appears; cannot say at the debrief what is gossiped and what is not, or why the removed backend came back	Finds the tombstone fix but not the clock; needs a nudge to look past "the backend should be idempotent"	Reasons from versions and invariants; finds B3 through the re-add; explains B1 from the points on the ring, not from the symptom; raises tie-breaks or tombstone expiry unprompted
Coding	Lets the AI add a dependency or rewrite a package; tests pass	Working fixes, but tests on a single node or a single key, or	Small diffs in the style of the repository; each regression test

Competency	Weak	Borderline	Strong
	before the fix too; never runs -race	time.Sleep where a clock belongs	fails first; two-node tests for gossip; deterministic and race-free
Effective communication	Prompts like "the backend comes back, fix it"; accepts "make the service idempotent" as the answer to RLY-31; decisions made silently by the AI	Explains what they are doing but leaves the retry policy or the re-add semantics implicit	Prompts carry the commands, the output and the expectation; challenges a wrong claim with a curl; states the retry policy and asks about it; the incident note is clear and does not shift blame
Speed and getting to a result	Applies AI suggestions one after another until the output looks right; never starts the cluster	Gets there, but slower than the time boxes because checks come late	Reproduces before fixing, verifies after, and still finishes Tasks 1 and 3 and the chosen feature in time

Fill in the bug sheet as you go. It is not a score; it is the evidence the ratings point back to.

	Found by	Root cause fixed?	Test failed before fix?	Notes
B1	candidate / AI / no			
B2	candidate / AI / no			
B3	candidate / AI / no			
B4	candidate / AI / no			
B5	candidate / AI / no			
B6	candidate / AI / no			

Also record three plain counts per task: **files the candidate opened**, **files the AI opened**, and **AI claims accepted without checking**. And note when the candidate asked the AI to "find all the bugs": that is a legitimate move. What counts is what they did with the list, which on this codebase usually mixes real bugs with confident false ones.

Collecting data

Collect the git history with the task-N tags and the final working tree; the AI transcript (a Claude Code session export, the Cursor chat, or the text of the chat window); the screen recording with the candidate's audio; and the interviewer's sheet with timestamped notes and the bug sheet.

Reference fixes

A short version for the interviewer. With all six applied, every existing test passes under -race, and a regression test for each bug fails before its fix and passes after it.

B1 · walk the whole ring

```
// ring.go, in Lookup
for n := range len(r.points) {
```

Every point is visited at most once, so the walk ends; it stops at the first usable backend, which in practice is a few points away.

B2 · "in a row" means a success resets the failures

```
// health.go, in Record
if ok {
    s.failures = 0
    s.successes++
```

B3 · advance the clock on receive

```
// cluster.go, in Merge
for _, e := range remote {
    s.clock = max(s.clock, e.Version)
    cur, ok := s.entries[e.Backend]
```

For every received entry, not only the ones that win. A tie-break for equal versions is a good addition, not required.

B4 · retry only what is safe to repeat

```
// proxy.go, in ServeHTTP
if err != nil {
    if last || !(idempotent(r.Method) || notSent(err)) { ... 502 ... }
    continue
}
if resp.StatusCode >= 500 && !last && idempotent(r.Method) { ... continue }

func idempotent(method string) bool {
    switch method {
    case http.MethodGet, http.MethodHead, http.MethodOptions:
        return true
    }
    return false
}

// notSent reports whether the request never left: the connection was refused.
func notSent(err error) bool {
    var op *net.OpError
    return errors.As(err, &op) && op.Op == "dial"
}
```

PUT and DELETE are idempotent by the HTTP specification; whether to trust that of the service behind Relay is a fair question to ask the candidate.

B5 · the client's IP, appended

```
func clientIP(r *http.Request) string {
    host, _, err := net.SplitHostPort(r.RemoteAddr)
    if err != nil {
        return r.RemoteAddr
    }
    return host
}

// ServeHTTP:  key = clientIP(r)
// forward:
xff := clientIP(r)
if prior := r.Header.Get("X-Forwarded-For"); prior != "" {
    xff = prior + ", " + xff
}
out.Header.Set("X-Forwarded-For", xff)
```

The key must not come from the incoming `X-Forwarded-For`: any client can write that header and pick its own backend.

B6 · tombstones

```
// cluster.go
Removed Status = "removed"

func (s *State) Remove(backend string) bool {
    s.mu.Lock()
    cur, ok := s.entries[backend]
    ok = ok && cur.Status != Removed
    if ok {
        s.clock++
        s.entries[backend] = Entry{Backend: backend, Status: Removed, Version: s.clock}
    }
    s.mu.Unlock()
    ...
}
```

`Backends` leaves tombstones out, so they are not health-checked; `ActiveBackends` already does. `Set` still refuses `removed` from the admin API. Re-adding through another node works only together with B3.

Showing each bug on the cluster

For the prep machine, to see each bug before the session, and for checking a candidate's claim during it. Start the cluster with examples/cluster.sh in one terminal and run these in another. Reset a backend with curl -X PUT localhost:900N/control -d '{}', and restart the cluster between sections, because gossip keeps what the previous section changed. Every command and output here comes from a real run.

A helper that prints a node's view in one line:

```
st() { curl -s localhost:$1/relay/status | python3 -c 'import json,sys; d=json.load(sys.stdin); print(d["node"], " ".join("%s:%s@v%d%s" % (b["backend"][-4:], b["status"], b["version"], "" if b["healthy"] else "(down)") for b in d["backends"]))'; }
```

B6 · the removed backend comes back

```
curl -X DELETE localhost:8001/relay/backends/127.0.0.1:9003     # 204
sleep 2; st 8001
relay-1 9001:active@v0 9002:active@v0 9003:active@v0
```

B3 · a drain through relay-2 is reverted

```
curl -X PUT localhost:8001/relay/backends/127.0.0.1:9002 -d '{"status":"draining"}'   # version 1
curl -X PUT localhost:8001/relay/backends/127.0.0.1:9002 -d '{"status":"active"}'     # version 2
sleep 2
curl -X PUT localhost:8002/relay/backends/127.0.0.1:9002 -d '{"status":"draining"}'   # version 1 again
sleep 2; st 8002
relay-2 9001:active@v0 9002:active@v2 9003:active@v0
```

B5 · no stickiness without a key

```
for i in 1 2 3 4 5 6; do curl -s localhost:8001/hello; echo; done
```

The backend field changes from call to call, and x_forwarded_for reads 127.0.0.1:47794, a new port every time. Adding -H 'X-Forwarded-For: 203.0.113.7' shows the incoming value is replaced, not appended.

B1 · 503 while two backends are healthy

```
curl -X PUT localhost:9002/control -d '{"fail_health": true}'
sleep 4
curl -s -H 'X-Relay-Key: user-157' localhost:8001/x            # no backend available
for i in $(seq 0 999); do
  curl -s -o /dev/null -w '%{http_code}\n' -H "X-Relay-Key: user-$i" localhost:8001/x
done | sort | uniq -c                                          # 984 x 200, 16 x 503
```

user-157 fails because of the addresses 127.0.0.1:9001 to 9003; on other ports, other keys fail.

B2 · one blip ejects only the backend that had trouble before

```
curl -X PUT localhost:9002/control -d '{"fail_health": true}'; sleep 6
curl -X PUT localhost:9002/control -d '{}'; sleep 4; st 8001          # all up again
for p in 9002 9003; do curl -X PUT localhost:$p/control -d '{"fail_health": true}'; done
sleep 1
for p in 9002 9003; do curl -X PUT localhost:$p/control -d '{}'; done
st 8001
relay-1 9001:active@v0 9002:active@v0(down) 9003:active@v0
```

If neither drops, the one-second blip missed a health check; repeat the last three steps.

B4 · one POST, two orders

```
curl -X PUT localhost:9002/control -d '{"fail_rate": 1}'
curl -si -X POST -H 'X-Relay-Key: alice' localhost:8001/orders -d '{"item":"book"}'
   # 200, X-Relay-Backend: 127.0.0.1:9001
curl -s localhost:9001/orders; curl -s localhost:9002/orders    # the order is on both
```

alice belongs to 9002 on these addresses. With fail_rate 0.5 and 30 keys, as in the Task 3 replay, our run gave 5 duplicates.

Caveats

- Verified with Go 1.27.3 on Linux, three nodes and three backends on 127.0.0.1. The share of keys that B1 drops (1–6% with one of three backends down) depends on the backend addresses, because they are what is hashed; on other ports the failing keys differ. Find one on the prep machine if you want to show it.

- The B2 replay on the cluster depends on a one-second blip landing on a health check, which happens most of the time; a unit test with `Record` reproduces it every time.
- The demo backend fails **after** doing the work on purpose; that is what makes B4 visible. A backend that fails before doing anything would hide it.
- The code is small enough that an agentic AI can read all of it in one pass. That is intended: the session is about verification, not search. Expect "find the bugs" prompts and judge what follows.
- The code lives in a public repository. If a candidate could plausibly have seen it, prefer another session.